Linked List

Dr. Nafis Sadeq
Assistant Professor
Department of Computer Science and Engineering
East West University



Today's Plan

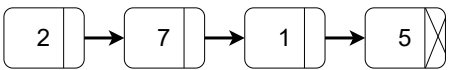
1. Linked List Introduction

Linked List

- A linked list is a linear collection of data elements (called nodes), where the linear order is given by means of pointers.
- Each node is divided into 2 parts:
  - 1st part contains the information of the element.
  - 2nd part is called the **link field** or **next pointer field** which contains the address of the next node in the list.

```
class Node {
public:
    int value;
    Node* next;

    Node(int val) {
        value = val;
        next = nullptr;
    }
};
```



Basic Operations

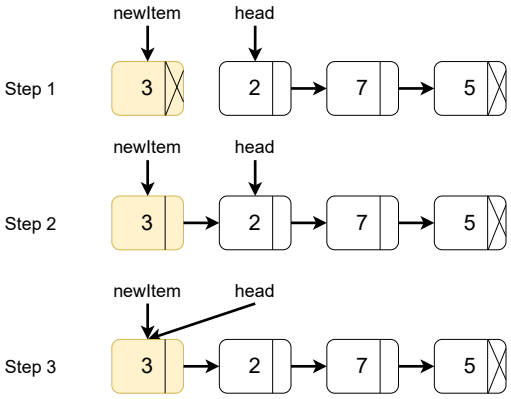
- Insert: Add a new node in the first, last or interior of the list.
- Delete: Delete a node from the first, last or interior of the list.
- Search: Search a node containing particular value in the linked list.

Insert First

- To add a new node to the head of the linear linked list, we need to construct a new node that is pointed by pointer *newitem*.
- Assume there is a global variable *head* which points to the first node in the list.
- The new node points to the first node in the list. The *head* is then set to point to the new node.

Insert First (Cont.)

- **Step 1**. Create a new node that is pointed by pointer *newItem*.
- **Step 2**. Link the new node to the first node of the linked list.
- **Step 3**. Set the pointer *head* to the new node.

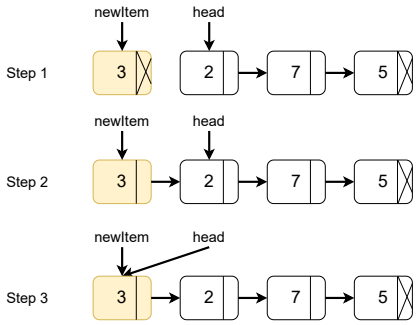


Insert First (Cont.)

```
class Node {
public:
    int value;
    Node* next;

    Node(int v){
        value = v;
        next = nullptr;
    }
};

void insertHead(Node*& head, int value) {
    Node* newItem = new Node(value);
    newItem->next = head;
    head = newItem;
}
```

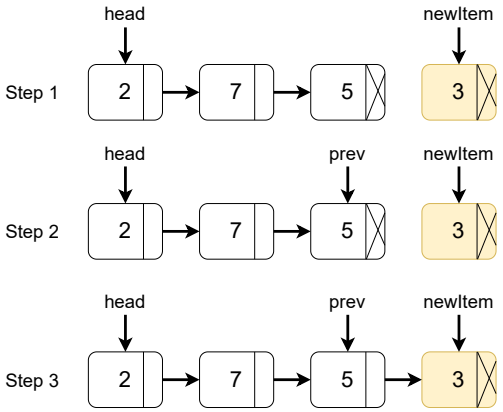


Insert Last

- To add a new node to the tail of the linear linked list, we need to construct a new node and set its link field to `NULL`.
- Assume the list is not empty; locate the last node and change its link field to point to the new node.

Insert Last (Cont.)

- **Step 1**. Create the new node.
- **Step 2**. Set a temporary pointer *prev* to point to the last node.
- **Step 3**. Set *prev* to point to the new node and make the new node the last node.



Insert Last (Cont.)

```
class Node {
public:
    int value;
    Node* next;

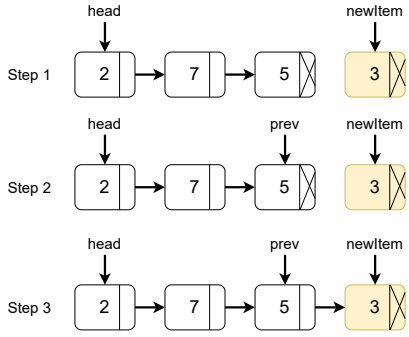
    Node(int v) {
        value = v;
        next  = nullptr;
    }
};

void insertTail(Node*& head, int value) {

    Node* newItem = new Node(value);
    if (head == nullptr) {
        head = newItem;
        return;
    }

    Node* prev = head;
    while (prev->next != nullptr) {
        prev = prev->next;
    }

    prev->next = newItem;
}
```

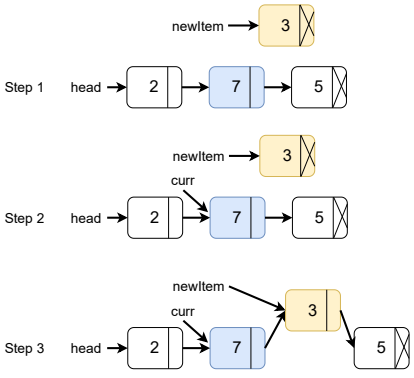


Insert After (Middle)

- To add a new node after a desired node (identified by its value), construct a new node.
- Assume the list is not empty and the desired value exists; locate that node and adjust links to place the new node *after* it.

Insert After (Cont.)

- **Step 1**. Create the new node.
- **Step 2**. Set a temporary pointer *curr* to the node whose value equals the target value.
- **Step 3**. Link the new node after *curr* and link next pointer of the new node to the node previously next to *curr*.



Insert After (Cont.)

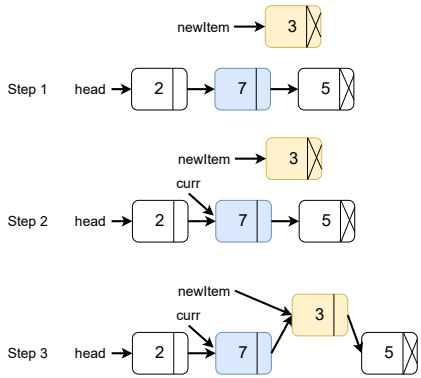
```
class Node {
public:
    int value;
    Node* next;

    Node(int v) {
        value = v;
        next  = nullptr;
    }
};

void insertAfter(Node*& head, int key, int value) {
    if (head == nullptr) return;

    Node* curr = head;
    while (curr != nullptr && curr->value != key) {
        curr = curr->next;
    }
    if (curr == nullptr) return;

    Node* newItem = new Node(value);
    newItem->next = curr->next;
    curr->next    = newItem;
}
```

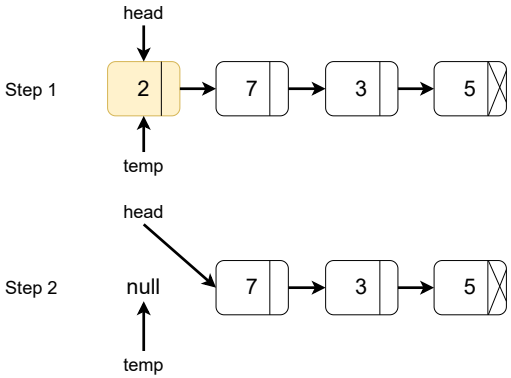


Delete First

- To delete the first node from a linear linked list, move the *head* pointer to the second node.
- If the list is empty, no deletion is performed.
- After updating *head*, deallocate the removed node.

Delete First (Cont.)

- **Step ***. If the list is empty, return.
- **Step 1**. Set a temporary pointer *temp* to the first node.
- **Step 2**. Move *head* to *head*→*next* and delete *temp*.



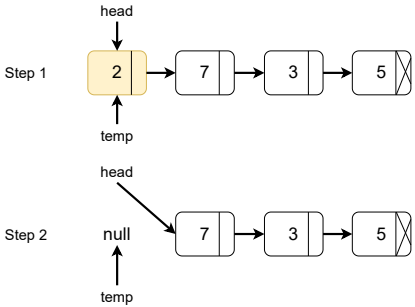
Delete First (Cont.)

```
class Node {
public:
    int value;
    Node* next;

    Node(int v){
        value = v;
        next = nullptr;
    }
};

void deleteFirst(Node*& head) {
    if (head == nullptr) return;

    Node* temp = head;
    head = head->next;
    delete temp;
}
```

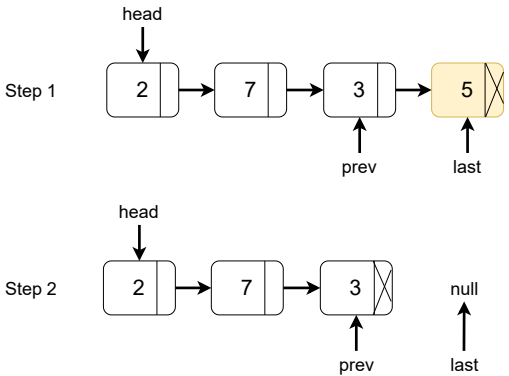


Delete Last

- To delete the last node, locate the *second last* node and set its link field to `NULL`.
- Handle the special case when the list has only one node: set *head* to `NULL`.
- Deallocate the removed last node.

Delete Last (Cont.)

- **Step ***. If the list is empty, return.
- **Step ***. If there is only one node, delete it and set *head* to `NULL`.
- **Step 1**. Otherwise, traverse with pointer *prev* until *prev*→*next* is the last node.
- **Step 2**. Delete the last node and set *prev*→*next* to `NULL`.



Delete Last (Cont.)

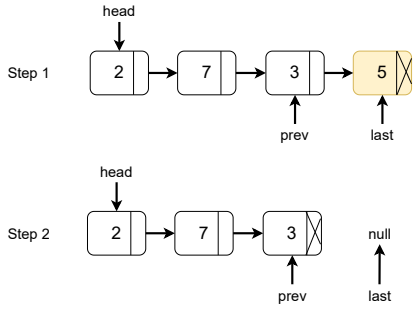
```
class Node {
public:
    int value;
    Node* next;

    Node(int v){
        value = v;
        next = nullptr;
    }
};

void deleteLast(Node*& head) {
    if (head == nullptr) return;
    if (head->next == nullptr) {
        delete head;
        head = nullptr;
        return;
    }

    Node* prev = head;
    while (prev->next->next != nullptr) {
        prev = prev->next;
    }

    Node* last = prev->next;
    prev->next = nullptr;
    delete last;
}
```

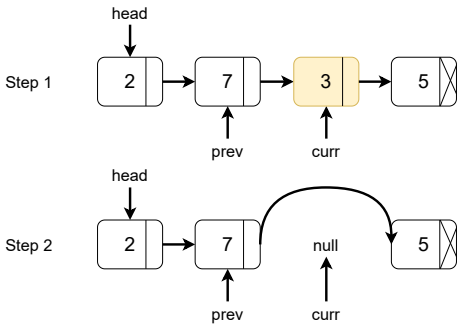


Delete Any (By Value)

- To delete a node with a given key value, locate the first node whose value equals the key.
- If the node is at the head, update *head*; otherwise link *prev* around *curr*.
- Deallocate the removed node. If the key is not found, no deletion is performed.

Delete Any (Cont.)

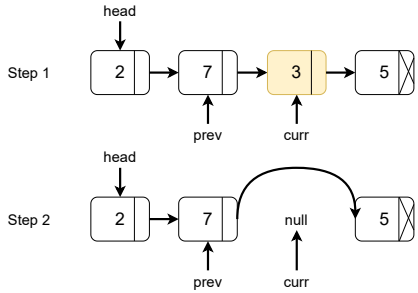
- **Step ***. If the list is empty, return.
- **Step ***. If *head→value == key*, remove the first node.
- **Step 1**. Otherwise, traverse with pointers *prev* and *curr* until *curr→value == key*.
- **Step 2**. Relink: set *prev→next = curr→next* and delete *curr*.



Delete Any (Cont.)

```
class Node {
public:
    int value;
    Node* next;
    Node(int v){
        value = v;
        next = nullptr;
    }
};

void deleteValue(Node*& head, int key) {
    if (head == nullptr) return;
    if (head->value == key) {
        Node* temp = head;
        head = head->next;
        delete temp;
        return;
    }
    Node* prev = head;
    Node* curr = head->next;
    while (curr != nullptr && curr->value != key) {
        prev = curr;
        curr = curr->next;
    }
    if (curr == nullptr) return;
    prev->next = curr->next;
    delete curr;
}
```



Introduction: Doubly Linked List

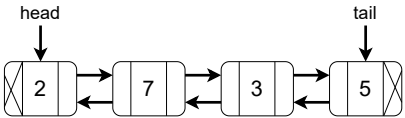
- A doubly linked list stores elements in nodes, each with *data*, a pointer to the *next* node, and a pointer to the *previous* node.
- Supports bidirectional traversal and $O(1)$ insert/delete when a node position is known.
- Requires extra memory for the additional pointer and careful pointer updates.

Implementation of Doubly Linked List

```
class DNode {
public:
    int value;
    DNode* prev;
    DNode* next;

    DNode(int v){
        value = v;
        prev = nullptr;
        next = nullptr;
    }
};

DNode* head = nullptr;
DNode* tail = nullptr;
```



Basic Operations in Doubly Linked List

- Insertion: at head, at tail, in the middle (after a given key)
- Deletion: at head, at tail, in the middle (by value)
- Search and traversal: forward and backward

Insert Middle in Doubly Linked List

- To insert after a desired node identified by value, construct a new node.
- Locate the node with the target value and splice the new node after it.
- Update up to four pointers: of the new node, its predecessor, and its successor; update *tail* if needed.

Insert Middle in Doubly Linked List (Cont.)

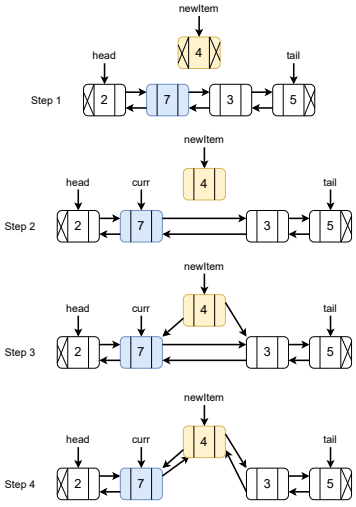
- **Step 1**. Create the new node with the given value.
- **Step 2**. Find the node *curr* whose value equals the key.
- **Step 3**. Set `newItem->prev = curr` and `newItem->next = curr->next`.
- **Step 4**. If `curr->next` exists, set `curr->next->prev = newItem`; set `curr->next = newItem`; update *tail* when inserting at the end.

Insert Middle in Doubly Linked List (Cont.)

```
class DNode {
public:
    int value;
    DNode* prev;
    DNode* next;

    DNode(int v){
        value = v;
        prev = nullptr;
        next = nullptr;
    }
};

void insertAfter(DNode*& head, DNode*& tail, int key, int
     value) {
    if (head == nullptr) return;
    DNode* curr = head;
    while (curr != nullptr && curr->value != key) curr =
     curr->next;
    if (curr == nullptr) return;
    DNode* newItem = new DNode(value);
    newItem->prev = curr;
    newItem->next = curr->next;
    if (curr->next != nullptr) curr->next->prev = newItem;
    curr->next = newItem;
    if (tail == curr) tail = newItem;
}
```



Delete Middle in Doubly Linked List

- To delete a node by value in the middle, locate the first node with the key.
- Relink its predecessor and successor around it.
- Update *head* or *tail* when deleting the first or last node among matches.

Delete Middle in Doubly Linked List (Cont.)

- **Step ***. If the list is empty, return.
- **Step 2**. Find *curr* such that `curr->value == key`.
- **Step 3**. If `curr->prev` exists, set `curr->prev->next = curr->next`; else set `head = curr->next`.
- **Step 4**. If `curr->next` exists, set `curr->next->prev = curr->prev`; else set `tail = curr->prev`.
- **Step 5**. Deallocate the removed node.

Delete Middle in Doubly Linked List (Cont.)

```
class DNode {
public:
    int value;
    DNode* prev;
    DNode* next;

    DNode(int v){
        value = v;
        prev = nullptr;
        next = nullptr;
    }
};

void deleteValue(DNode*& head, DNode*& tail, int key) {
    if (head == nullptr) return;
    DNode* curr = head;
    while (curr != nullptr && curr->value != key) curr =
     curr->next;
    if (curr == nullptr) return;
    if (curr->prev != nullptr) curr->prev->next = curr->
     next;
    else head = curr->next;
    if (curr->next != nullptr) curr->next->prev = curr->
     prev;
    else tail = curr->prev;
    delete curr;
}
```

